Experiment No. 10

End-to-End Data Analytics Project: Business Intelligence and Predictive Analytics (Employee Attrition - IBM HR Dataset)

Aim

To implement a complete analytics workflow on a real-world dataset: data collection, preprocessing, exploratory analysis, statistical testing, visualisation, predictive modelling, evaluation, and presentation of actionable business insights.

Business Problem

A company loses about 16% of its employees every period. **Objective:** (1) find what drives attrition, and (2) build a model that flags employees at high risk of leaving so that HR can act early. Dataset: IBM HR Employee Attrition (1,470 employees, 35 attributes; target *Attrition* = Yes/No).

Methodology (Workflow)

Stage	Activity	Tools
1. Collect	Import the dataset (CSV)	pandas
2. Preprocess	Missing/duplicate check, drop constant and ID columns, IQR outlier capping, one-hot encoding, feature engineering (TenureRatio)	pandas, NumPy
3. EDA	Descriptive statistics, attrition rates by group	pandas
4. Statistics	Chi-square tests, Welch t-test, Pearson correlation	SciPy
5. Visualise	Six-panel dashboard	Matplotlib, Seaborn (Power BI/Tableau optional)
6. Model	Logistic Regression, Decision Tree, Random Forest (+ baseline); K-Means segmentation	scikit-learn
7. Evaluate	Accuracy, Precision, Recall, F1, ROC-AUC, confusion matrix, cross-validation	scikit-learn
8-10. Insights and report	Feature importance, risk ranking, segments, recommendations	This report

Program and Output

The complete program was run on the actual dataset (random_state = 42); green boxes show the real output and the figures are the real plots.

Steps 1-2: Collect data and preprocess

```
import pandas as pd
import numpy as np
import matplotlib
matplotlib.use("Agg")                      # remove this line in Jupyter/Colab
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.dummy import DummyClassifier
from sklearn.cluster import KMeans
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score,
```

```
                            roc_auc_score, confusion_matrix, classification_report, roc_curve,
                            silhouette_score)

pd.set_option("display.width", 120)
pd.set_option("display.max_columns", 20)
sns.set_theme(style="whitegrid")
RS = 42

# ---------- Step 1-2: collect data and preprocess ----------
# Business problem: predict which employees will leave (Attrition) and find the reasons.
raw = pd.read_csv("hr.csv", encoding="utf-8-sig")          # IBM HR Employee Attrition
print("Shape:", raw.shape)
print("Missing values:", int(raw.isnull().sum().sum()), "| Duplicate rows:", int(raw.duplicated().sum()))
const = [c for c in raw.columns if raw[c].nunique() == 1]
print("Constant columns dropped:", const, "+ EmployeeNumber (ID)")

df = raw.drop(columns=const + ["EmployeeNumber"]).drop_duplicates()
df["Attrition"] = (df["Attrition"] == "Yes").astype(int)      # target: 1 = left
df["TenureRatio"] = df["YearsAtCompany"] / (df["TotalWorkingYears"] + 1)   # feature engineering
print("Attrition rate: %.1f%%  (%d of %d)" % (df["Attrition"].mean() * 100, df["Attrition"].sum(), len(df)))

# outliers: IQR capping on skewed numeric columns (used for the modelling copy only)
df_model = df.copy()
for c in ["MonthlyIncome", "TotalWorkingYears", "YearsAtCompany", "YearsInCurrentRole",
          "YearsSinceLastPromotion", "YearsWithCurrManager"]:
    q1, q3 = df[c].quantile([0.25, 0.75]); iqr = q3 - q1
    lo, hi = q1 - 1.5 * iqr, q3 + 1.5 * iqr
    n = int(((df[c] < lo) | (df[c] > hi)).sum())
    df_model[c] = df[c].clip(lo, hi)
    print(f"  {c}: {n} outliers capped")

X = pd.get_dummies(df_model.drop(columns="Attrition"), drop_first=True, dtype=int)   # one-hot encoding
y = df_model["Attrition"]
print("Model matrix:", X.shape)
```

Output:

```
Shape: (1470, 35)
Missing values: 0 | Duplicate rows: 0
Constant columns dropped: ['EmployeeCount', 'Over18', 'StandardHours'] + EmployeeNumber (ID)
Attrition rate: 16.1%  (237 of 1470)
  MonthlyIncome: 114 outliers capped
  TotalWorkingYears: 63 outliers capped
  YearsAtCompany: 104 outliers capped
  YearsInCurrentRole: 21 outliers capped
  YearsSinceLastPromotion: 107 outliers capped
  YearsWithCurrManager: 14 outliers capped
Model matrix: (1470, 45)
```

Step 3: Exploratory data analysis

```
# ---------- Step 3: exploratory data analysis ----------
print("\nMedians by Attrition (0 = stayed, 1 = left):")
print(df.groupby("Attrition")[["Age", "MonthlyIncome", "YearsAtCompany", "TotalWorkingYears"]].median())
for c in ["OverTime", "MaritalStatus", "BusinessTravel"]:
    print(f"\nAttrition rate (%) by {c}:")
    print((df.groupby(c)["Attrition"].mean() * 100).round(1).sort_values(ascending=False).to_string())
jr = (df.groupby("JobRole")["Attrition"].mean() * 100).round(1).sort_values(ascending=False)
print("\nHighest / lowest attrition JobRole (%):\n", jr.head(2).to_string(), "\n", jr.tail(2).to_string())
piv = df.pivot_table(values="Attrition", index="JobLevel", columns="OverTime", aggfunc="mean") * 100
print("\nAttrition rate (%) by JobLevel x OverTime:\n", piv.round(1))
```

Output:

```
Medians by Attrition (0 = stayed, 1 = left):
             Age  MonthlyIncome  YearsAtCompany  TotalWorkingYears
Attrition
0           36.0         5204.0             6.0               10.0
1           32.0         3202.0             3.0                7.0

Attrition rate (%) by OverTime:
OverTime
Yes    30.5
No     10.4

Attrition rate (%) by MaritalStatus:
MaritalStatus
Single      25.5
Married     12.5
Divorced    10.1

Attrition rate (%) by BusinessTravel:
BusinessTravel
```

```
Travel_Frequently    24.9
Travel_Rarely        15.0
Non-Travel            8.0

Highest / lowest attrition JobRole (%):
 JobRole
Sales Representative     39.8
Laboratory Technician    23.9
 JobRole
Manager              4.9
Research Director    2.5

Attrition rate (%) by JobLevel x OverTime:
 OverTime    No   Yes
JobLevel
1         15.8  52.6
2          6.7  17.8
3         12.3  20.6
4          2.7   9.1
5          3.9  16.7
```

Step 4: Statistical analysis

```
# ---------- Step 4: statistical analysis ----------
print("Chi-square tests (H0: variable independent of Attrition):")
for c in ["OverTime", "MaritalStatus", "BusinessTravel", "JobRole", "Department", "Gender"]:
    ct = pd.crosstab(df[c], df["Attrition"])
    chi2, p, dof, _ = stats.chi2_contingency(ct)
    v = np.sqrt(chi2 / (ct.values.sum() * (min(ct.shape) - 1)))
    print(f"  {c:15s} chi2={chi2:7.2f}  p={p:.3g}  Cramer's V={v:.2f}  ->", "significant" if p < 0.05 else "not significant

left, stay = df.loc[df.Attrition == 1, "MonthlyIncome"], df.loc[df.Attrition == 0, "MonthlyIncome"]
t, p = stats.ttest_ind(left, stay, equal_var=False)
print(f"\nWelch t-test MonthlyIncome (left vs stayed): means {left.mean():.0f} vs {stay.mean():.0f}, t={t:.2f}, p={p:.3g}"
corr = df.select_dtypes("number").corr()["Attrition"].drop("Attrition").sort_values()
print("\nStrongest Pearson correlations with Attrition:")
print(pd.concat([corr.head(4), corr.tail(2)]).round(3).to_string())
```

Output:

```
Chi-square tests (H0: variable independent of Attrition):
  OverTime        chi2=  87.56  p=8.16e-21  Cramer's V=0.24  -> significant
  MaritalStatus   chi2=  46.16  p=9.46e-11  Cramer's V=0.18  -> significant
  BusinessTravel  chi2=  24.18  p=5.61e-06  Cramer's V=0.13  -> significant
  JobRole         chi2=  86.19  p=2.75e-15  Cramer's V=0.24  -> significant
  Department      chi2=  10.80  p=0.00453  Cramer's V=0.09  -> significant
  Gender          chi2=   1.12  p=0.291  Cramer's V=0.03  -> not significant

Welch t-test MonthlyIncome (left vs stayed): means 4787 vs 6833, t=-7.48, p=4.43e-13

Strongest Pearson correlations with Attrition:
TotalWorkingYears    -0.171
JobLevel             -0.169
YearsInCurrentRole   -0.161
MonthlyIncome        -0.160
NumCompaniesWorked    0.043
DistanceFromHome      0.078
```

Step 5: Visualisation dashboard

```
# ---------- Step 5: visualisation dashboard ----------
fig, ax = plt.subplots(2, 3, figsize=(15, 8))
ax[0, 0].pie(df["Attrition"].value_counts().sort_index(), labels=["Stayed", "Left"], autopct="%1.1f%%",
             colors=["#9ecae1", "#e6550d"]); ax[0, 0].set_title("Attrition share")
r = df.groupby("OverTime")["Attrition"].mean() * 100
sns.barplot(x=r.index, y=r.values, ax=ax[0, 1], hue=r.index, palette="Reds", legend=False)
ax[0, 1].set_title("Attrition rate (%) by OverTime"); ax[0, 1].set_ylabel("%")
sns.barplot(x=jr.values, y=jr.index, ax=ax[0, 2], color="#e6550d"); ax[0, 2].set_title("Attrition rate (%) by JobRole")
sns.boxplot(x="Attrition", y="MonthlyIncome", data=df, ax=ax[1, 0], hue="Attrition", palette="Set2", legend=False)
ax[1, 0].set_title("Monthly income: stayed (0) vs left (1)")
sns.histplot(data=df, x="Age", hue="Attrition", kde=True, ax=ax[1, 1], stat="density", common_norm=False)
ax[1, 1].set_title("Age distribution by Attrition")
sns.heatmap(piv, annot=True, fmt=".1f", cmap="Reds", ax=ax[1, 2]); ax[1, 2].set_title("Attrition % by JobLevel x OverTime"
plt.tight_layout(); plt.savefig("f10_dash.png", dpi=105); plt.show()
```

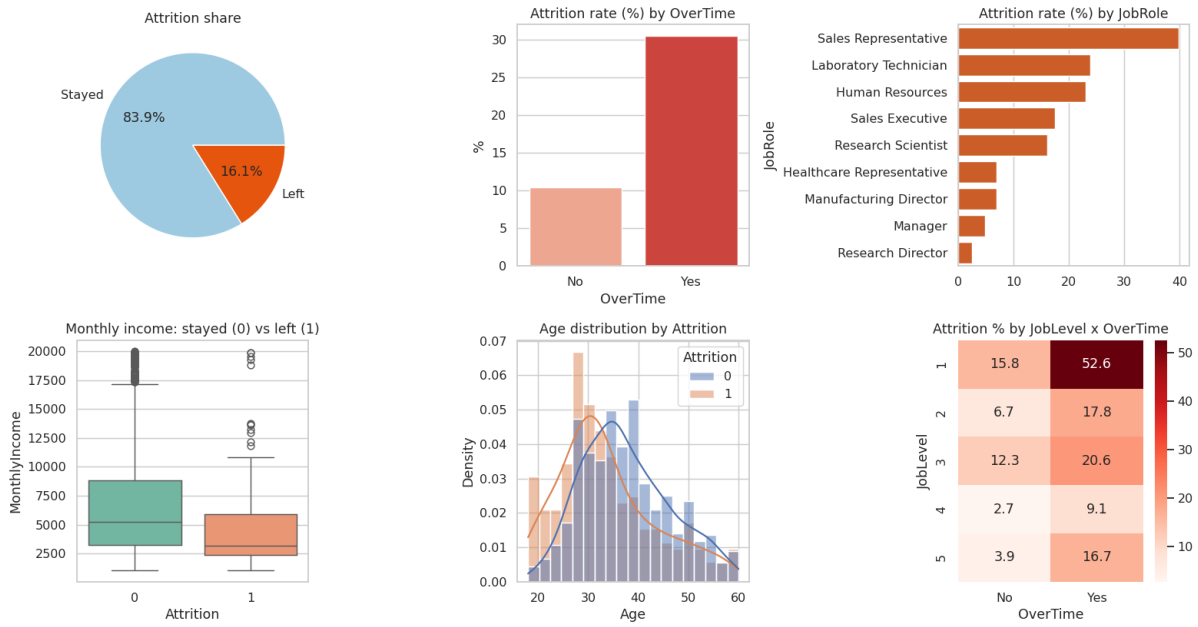


Fig. 1: Attrition dashboard

Step 6: Predictive modelling

```
# ---------- Step 6: predictive modelling ----------
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, stratify=y, random_state=RS)
print("Train:", X_tr.shape, "| Test:", X_te.shape, "| test attrition rate: %.1f%%" % (y_te.mean() * 100))

models = {
    "Baseline (majority)": DummyClassifier(strategy="most_frequent"),
    "Logistic Regression": make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000, class_weight="balanced", C=0.1
    "Decision Tree": DecisionTreeClassifier(max_depth=5, min_samples_leaf=20, class_weight="balanced", random_state=RS),
    "Random Forest": RandomForestClassifier(n_estimators=300, max_depth=8, min_samples_leaf=5,
                                            class_weight="balanced", random_state=RS, n_jobs=-1),
}
cv = StratifiedKFold(5, shuffle=True, random_state=RS)
rows, probs = [], {}
for name, m in models.items():
    m.fit(X_tr, y_tr)
    pred = m.predict(X_te); pr = m.predict_proba(X_te)[:, 1]; probs[name] = pr
    cv_auc = cross_val_score(m, X_tr, y_tr, cv=cv, scoring="roc_auc").mean()
    rows.append([name, accuracy_score(y_te, pred), precision_score(y_te, pred, zero_division=0),
                 recall_score(y_te, pred), f1_score(y_te, pred), roc_auc_score(y_te, pr), cv_auc])
```

Output:

```
Train: (1176, 45) | Test: (294, 45) | test attrition rate: 16.0%
```

Step 7: Model evaluation

```
# ---------- Step 7: model evaluation ----------
res = pd.DataFrame(rows, columns=["Model", "Accuracy", "Precision", "Recall", "F1", "ROC-AUC", "CV AUC"]).set_index("Model
print(res.round(3).to_string())

best = res.drop("Baseline (majority)")["ROC-AUC"].idxmax()
print("\nBest model by ROC-AUC:", best)
pred = models[best].predict(X_te)
print(classification_report(y_te, pred, target_names=["Stayed", "Left"], digits=3))
cm = confusion_matrix(y_te, pred)
print("Confusion matrix [[TN FP] [FN TP]]:\n", cm)

fig, ax = plt.subplots(1, 2, figsize=(11, 4.3))
for name in list(models)[1:]:
    fpr, tpr, _ = roc_curve(y_te, probs[name]); ax[0].plot(fpr, tpr, label=f"{name} (AUC {res.loc[name, 'ROC-AUC']:.2f})")
ax[0].plot([0, 1], [0, 1], "k--"); ax[0].set_xlabel("False positive rate"); ax[0].set_ylabel("True positive rate")
ax[0].set_title("ROC curves"); ax[0].legend(loc="lower right")
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", ax=ax[1], xticklabels=["Stayed", "Left"], yticklabels=["Stayed", "Left"
ax[1].set_title(f"Confusion matrix - {best}"); ax[1].set_xlabel("Predicted"); ax[1].set_ylabel("Actual")
plt.tight_layout(); plt.savefig("f10_eval.png", dpi=105); plt.show()
```

Output:

```
                    Accuracy  Precision  Recall     F1  ROC-AUC  CV AUC
Model
Baseline (majority)     0.840      0.000   0.000  0.000    0.500   0.500
```

```
Logistic Regression     0.769      0.376   0.681  0.485    0.801   0.831
Decision Tree           0.769      0.348   0.511  0.414    0.716   0.705
Random Forest           0.823      0.419   0.277  0.333    0.775   0.801

Best model by ROC-AUC: Logistic Regression
              precision    recall  f1-score   support

      Stayed      0.928     0.785     0.851       247
        Left      0.376     0.681     0.485        47

    accuracy                          0.769       294
   macro avg      0.652     0.733     0.668       294
weighted avg      0.840     0.769     0.792       294

Confusion matrix [[TN FP] [FN TP]]:
 [[194  53]
 [ 15  32]]
```

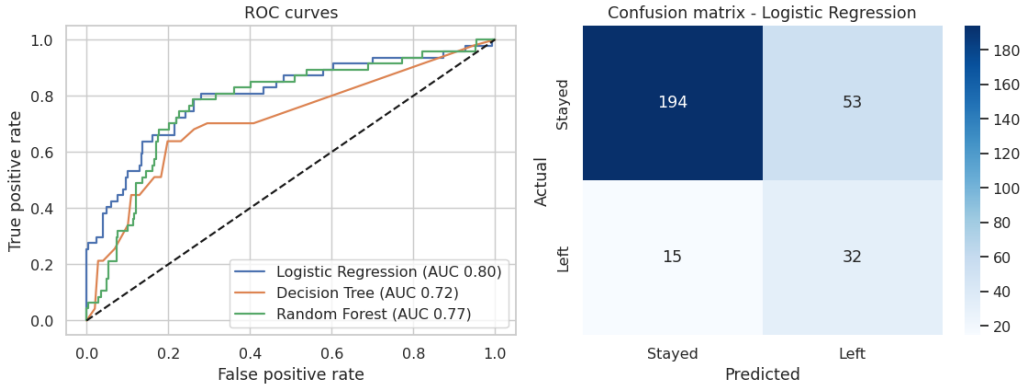


Fig. 2: ROC curves and confusion matrix of the best model

Step 8: Insights - feature importance, risk ranking and segmentation

```
# ---------- Step 8: insights - feature importance, risk ranking, segmentation ----------
rf = models["Random Forest"]
imp = pd.Series(rf.feature_importances_, index=X.columns).sort_values(ascending=False).head(10)
print("Top 10 Random Forest feature importances:\n", imp.round(3).to_string())
lr = models["Logistic Regression"]
coef = pd.Series(lr[-1].coef_[0], index=X.columns).sort_values()
print("\nLogistic regression - strongest coefficients (per 1 SD):")
print(pd.concat([coef.head(4), coef.tail(4)]).round(2).to_string())

fig, ax = plt.subplots(figsize=(7, 4))
imp[::-1].plot.barh(ax=ax, color="#e6550d"); ax.set_title("Top 10 drivers of attrition (Random Forest)")
plt.tight_layout(); plt.savefig("f10_imp.png", dpi=105); plt.show()

# risk ranking: how well does the model prioritise employees?
risk = pd.DataFrame({"p": probs[best], "y": y_te.values}).sort_values("p", ascending=False)
k = int(len(risk) * 0.2)
print(f"\nTop 20% highest-risk employees ({k}): actual attrition = {risk.head(k)['y'].mean()*100:.1f}% "
      f"vs overall {y_te.mean()*100:.1f}%; captures {risk.head(k)['y'].sum()/y_te.sum()*100:.1f}% of all leavers")

# K-Means segmentation of employees
seg_cols = ["Age", "MonthlyIncome", "TotalWorkingYears", "YearsAtCompany"]
Z = StandardScaler().fit_transform(df_model[seg_cols])
for k_ in (2, 3, 4):
    print(f"k={k_}: silhouette = {silhouette_score(Z, KMeans(k_, n_init=10, random_state=RS).fit_predict(Z)):.3f}")
df["Segment"] = KMeans(3, n_init=10, random_state=RS).fit_predict(Z)
seg = df.groupby("Segment").agg(n=("Attrition", "size"), age=("Age", "mean"), income=("MonthlyIncome", "mean"),
                                tenure=("YearsAtCompany", "mean"), attrition_pct=("Attrition", lambda s: s.mean() * 100))
print(seg.round(1).sort_values("attrition_pct", ascending=False).to_string())
```

Output:

```
Top 10 Random Forest feature importances:
 MonthlyIncome           0.076
Age                     0.067
TotalWorkingYears       0.064
OverTime_Yes            0.057
YearsAtCompany          0.047
DailyRate               0.047
TenureRatio             0.046
StockOptionLevel        0.044
YearsWithCurrManager    0.042
DistanceFromHome        0.041
```

```
Logistic regression - strongest coefficients (per 1 SD):
TotalWorkingYears                -0.49
EnvironmentSatisfaction          -0.38
JobSatisfaction                  -0.36
YearsWithCurrManager             -0.32
YearsSinceLastPromotion           0.42
BusinessTravel_Travel_Frequently  0.55
JobRole_Laboratory Technician     0.55
OverTime_Yes                      0.70

Top 20% highest-risk employees (58): actual attrition = 44.8% vs overall 16.0%; captures 55.3% of all leavers
k=2: silhouette = 0.466
k=3: silhouette = 0.306
k=4: silhouette = 0.330
           n   age   income  tenure  attrition_pct
Segment
0        567  29.6   3541.1     3.3           25.0
1        657  39.2   5870.2     7.6           11.6
2        246  47.6  15019.6    14.2            7.7
```

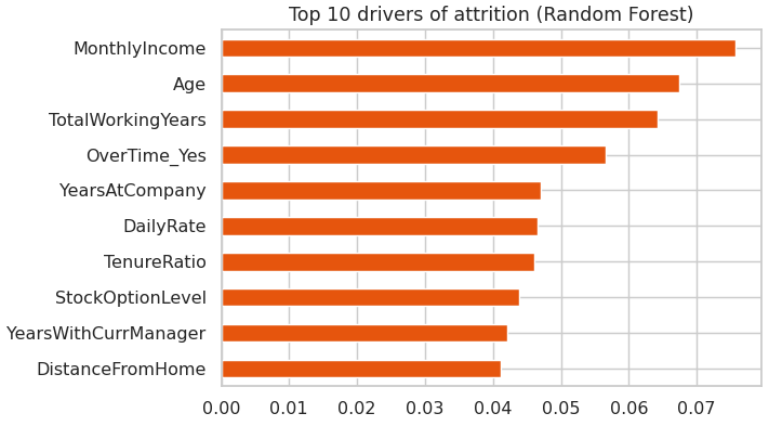


Fig. 3: Top drivers of attrition (Random Forest importance)

Results and Interpretation (Step 8)

Data and statistics

- Clean data: 1,470 rows, no missing values or duplicates. Attrition is imbalanced: 237 leavers (16.1%) vs 1,233 stayers.
- Chi-square tests show attrition is significantly associated with OverTime (p = 8.16e-21, Cramer's V = 0.24), JobRole (p = 2.75e-15, V = 0.24), MaritalStatus (p = 9.46e-11), BusinessTravel (p = 5.61e-06) and Department (p = 0.0045). **Gender is not significant** (p = 0.291).
- Welch t-test: leavers earn less (mean 4,787 vs 6,833; t = -7.48, p = 4.43e-13). Correlations with attrition are individually weak (strongest: TotalWorkingYears -0.171, JobLevel -0.169), so a combination of factors is needed for prediction.

Model comparison (20% stratified test set: 294 employees, 47 leavers)

Model	Accuracy	Precision	Recall	F1	ROC-AUC	CV AUC (5-fold)
Baseline (always 'stay')	0.840	0.000	0.000	0.000	0.500	0.500
Logistic Regression	0.769	0.376	**0.681**	**0.485**	**0.801**	**0.831**
Decision Tree (depth 5)	0.769	0.348	0.511	0.414	0.716	0.705
Random Forest	0.823	0.419	0.277	0.333	0.775	0.801

- Accuracy is misleading here: the baseline that never predicts a leaver scores 84% accuracy but catches no one (recall 0). Recall, F1 and ROC-AUC are the right metrics.
- **Logistic Regression** was the best model by ROC-AUC (0.801 test, 0.831 cross-validated). It correctly identifies 32 of 47 leavers (recall 68.1%) but also flags 53 stayers (precision 37.6%): confusion matrix TN

194, FP 53, FN 15, TP 32.

- Random Forest has higher accuracy (82.3%) but misses most leavers at the default 0.5 threshold (recall 27.7%); its threshold can be lowered if recall matters more.
- **Risk ranking:** the 20% of test employees with the highest predicted risk (58 people) had an actual attrition rate of 44.8% (vs 16.0% overall) and contained 55.3% of all leavers, so the model is useful for prioritising retention effort.
- Limitation: the test set has only 47 leavers, so metrics are noisy (CV AUC is the more stable estimate); the model finds associations, not causes.

Key drivers

- **Logistic regression** (per 1 SD): higher risk with OverTime (+0.70), Laboratory Technician role (+0.55), frequent travel (+0.55) and longer time since last promotion (+0.42); lower risk with more total working years (-0.49), better environment (-0.38) and job satisfaction (-0.36), and longer time with the current manager (-0.32).
- **Random Forest importance:** MonthlyIncome, Age, TotalWorkingYears, OverTime, YearsAtCompany, DailyRate, TenureRatio and StockOptionLevel lead.
- **EDA:** OverTime 30.5% vs 10.4% attrition; Single 25.5%; frequent travellers 24.9% vs 8.0% non-travellers; Sales Representatives 39.8% vs Research Directors 2.5%. Job level 1 employees working overtime leave at 52.6%.

K-Means segmentation (Age, MonthlyIncome, TotalWorkingYears, YearsAtCompany)

Segment	Employees	Avg age	Avg income	Avg tenure (yrs)	Attrition
0 - Young, low-paid, new	567	29.6	3,541	3.3	25.0%
1 - Mid-career	657	39.2	5,870	7.6	11.6%
2 - Senior, high-paid	246	47.6	15,020	14.2	7.7%

Note: the silhouette score was highest for k = 2 (0.466; k = 3 gave 0.306), so k = 3 was chosen mainly because three segments are easier to interpret for HR; the segment pattern (younger, lower-paid, shorter-tenure employees leave most) is clear either way.

Business Recommendations

- **Cut unsustainable overtime**, especially for job level 1 (52.6% attrition with overtime): rebalance workloads, hire, or compensate overtime.
- **Target the high-risk segment** (young, low-income, new joiners, 25% attrition): structured onboarding, mentoring, early promotion path and stock-option eligibility (no stock options: 24.4% attrition vs 7-9% at levels 1-2).
- **Review pay and workload** in Sales Representative and Laboratory Technician roles, and travel policies for frequent travellers.
- **Promotion and manager quality:** monitor time since last promotion and manager tenure; improve environment and job satisfaction scores.
- **Use the model as an early-warning tool**: review the top 20% risk list monthly (captures about 55% of leavers), pair it with stay interviews, and track the effect of interventions with A/B tests.
- **Ethical use:** use scores to offer support, not for punitive decisions; keep protected attributes such as gender out of decisions (it is not significant here anyway).

Presentation Outline (Steps 9-10)

- Slide 1: Problem and business impact (16% attrition).
- Slide 2: Data and preprocessing.
- Slide 3: Dashboard of key findings (Fig. 1).

- Slide 4: Statistical evidence (chi-square, t-test).
- Slide 5: Model comparison and ROC curves (Fig. 2).
- Slide 6: Drivers and employee segments (Fig. 3).
- Slide 7: Recommendations, expected benefits and next steps.

Optional: Power BI / Tableau dashboard

Export the cleaned data (with predicted risk) to CSV and load it in Power BI or Tableau. Suggested pages: KPI cards (headcount, attrition %, average income), attrition by department/job role/overtime/marital status, a tenure and age trend, slicers for department and job level, and a table of high-risk employees. The Python dashboard in Fig. 1 shows the same views.

Answers to Questions

Q1. Major stages of an end-to-end data analytics project.

- **Problem definition:** business objective and success metrics.
- **Data collection:** databases, files, APIs, web.
- **Data preprocessing:** missing values, duplicates, outliers, encoding, scaling, feature engineering.
- **Exploratory data analysis and statistical analysis:** distributions, correlations, hypothesis tests.
- **Visualisation and dashboards** for understanding and communication.
- **Modelling:** choose, train and tune algorithms. **Evaluation:** metrics, cross-validation.
- **Interpretation and recommendations**, then **deployment, monitoring and maintenance** with reporting to stakeholders.

Q2. Why is data preprocessing the foundation of analytics and ML?

Models learn only from the data they are given, so poor data leads to poor results (garbage in, garbage out). Preprocessing removes errors and duplicates, fills or removes missing values, controls outliers, converts categories to numbers and scales features so algorithms work properly and converge faster. In this project, constant and ID columns were removed (3 constant columns plus EmployeeNumber), extreme values in six skewed columns were capped by the IQR rule (114 + 63 + 104 + 21 + 107 + 14 = 423 values in total), and 45 clean numeric model features were produced.

Q3. How does EDA contribute to model development?

- Reveals data quality issues and the 16% class imbalance, so stratified splitting, class weights and recall/F1/AUC were used instead of accuracy.
- Shows which variables relate to the target (OverTime, JobLevel, income), guiding feature selection and engineering (TenureRatio).
- Shows skewness and outliers, guiding transformations and capping.
- Shows non-linear effects and interactions (JobLevel x OverTime), suggesting model type and features.
- Gives a sanity check and a baseline for judging whether model results are sensible.

Q4. Factors in selecting a machine learning algorithm.

Factor	Consideration
Problem type	Classification (churn/attrition), regression (price/sales), clustering (segments)
Data size and dimensionality	Small data: simple models; large data: ensembles and deep learning
Interpretability	Business may need explanations: logistic regression and trees are easier than boosting or neural networks
Data characteristics	Non-linearity, interactions, categorical features, missing values, class imbalance
Performance and metric	Choose by the metric that matches the business cost (recall for churn)
Training/prediction time and resources	Real-time needs, memory limits
Risk of overfitting and robustness	Regularisation, cross-validation

In this project, logistic regression was chosen because it gave the best ROC-AUC and is interpretable.

Q5. Why is model evaluation essential before deployment?

- Measures how well the model generalises to unseen data and detects overfitting (compare train/test and cross-validation).
- Shows whether it beats a simple baseline (always 'stay' gives 84% accuracy but zero recall).

- Uses metrics that match business costs (recall, precision, F1, ROC-AUC, MAE, RMSE, R^2) and reveals the types of errors (53 false alarms, 15 missed leavers).
- Allows fair comparison and selection among models; supports threshold choice.
- Prevents costly wrong decisions, bias and loss of trust after deployment.

Q6. How do BI tools complement machine learning?

- BI tools (Power BI, Tableau) present KPIs and trends interactively for non-technical users, while ML gives predictions and hidden patterns.
- ML output (risk scores, segments, forecasts) can be loaded into dashboards so managers can filter, drill down and act (e.g. list of high-risk employees by department).
- BI tools monitor the model and business KPIs after deployment (accuracy drift, attrition trend).
- They provide self-service reporting, scheduled refresh and sharing, which makes ML results usable across the organisation.

Q7. Challenges with real-world datasets.

- Missing, inconsistent, duplicated and noisy data; wrong data types and formats.
- Outliers and skewed distributions; class imbalance (16% positives here).
- Multiple sources that must be integrated; large volume and high dimensionality.
- Bias and unrepresentative samples; data leakage; changing data over time (drift).
- Privacy, security and ethical constraints (employee data); unclear business objectives; limited labelled data.
- Small test sets giving noisy metrics (47 leavers in this test set).

Q8. How can analytical insights be converted into actionable recommendations?

- Link each finding to a business goal and quantify impact (OverTime: 30.5% vs 10.4% attrition).
- Identify the controllable drivers (overtime, pay, travel, promotions) separately from uncontrollable ones (age).
- Propose specific actions with owners, cost and expected benefit (reduce overtime for level-1 staff; retention programme for the top 20% risk list).
- Prioritise by impact and feasibility, set KPIs and targets, pilot with A/B tests and measure results.
- Explain the limits (association is not causation) and review regularly.

Q9. Why are presentation and storytelling important?

- Decision-makers act on what they understand; a clear story (problem, evidence, insight, action) turns analysis into decisions.
- Good visuals and simple language make complex models and statistics accessible to non-technical stakeholders.
- Builds trust and buy-in, shows the business value and risks, and avoids misinterpretation.
- A project that is not communicated well is rarely implemented, however accurate the model is.

Q10. Future improvements and advanced techniques.

- **Better data:** more records and time-based data (performance reviews, engagement surveys, exit interviews, text analytics).
- **Modelling:** hyperparameter tuning (GridSearch/Optuna), gradient boosting (XGBoost, LightGBM, CatBoost), ensembles and stacking.
- **Imbalance handling:** SMOTE/over-sampling, cost-sensitive learning and decision-threshold tuning for the desired recall.
- **Explainability:** SHAP/LIME for individual-level reasons; probability calibration.
- **Survival analysis** to predict when an employee will leave; repeated cross-validation for more stable estimates.

- **Deployment:** API or dashboard integration, drift monitoring, scheduled retraining, A/B tests of retention actions, and fairness audits.

Conclusion

An end-to-end analytics project was completed on the IBM HR Attrition dataset. Preprocessing produced a clean, model-ready dataset; EDA and statistical tests showed that overtime, low job level and pay, frequent travel, single status and certain roles are linked to attrition (gender is not). A logistic regression model (ROC-AUC 0.80, recall 68%) outperformed a decision tree and random forest and ranked employees well: the top 20% risk group contained 55% of all leavers. K-Means showed that young, low-paid, short-tenure employees leave most (25% attrition). The findings support concrete HR actions and a monitored early-warning system.